WikipediaMoviesRetrieval

Assignment 1

Information Retrieval Course

Academic Year 2025/2026

Group Members:

Alperen Davran	[Student ID]
Matteo Comi	[Student ID]
Shakh [Surname]	[Student ID]
Amin Borqal	s0259707

University of Antwerp
Faculty of Science
Department of Computer Science

November 2, 2025

Contents

1 Introduction

This report outlines a compact information retrieval (IR) pipeline for the Wikipedia Movies dataset, designed to comply with the directives of the initial assignment in the IR course.

We delineate document tokenization, index construction, and query processing into distinct Python modules, organized within the `Components/` directory. Each document is stored as a concatenation of its `title` and `plot`, with the original title preserved in the metadata to ensure that retrieval outcomes remain interpretable throughout experimental processes.

1.1 System Architecture

The system is comprised of three modules, each with clearly defined interfaces:

1. **Tokenizer** — Receives as input: raw title and plot. Produces as output: a list of normalized tokens.
2. **Indexer** — Receives as input: a tuple of document identifier and tokens. Produces as output: an inverted index, either residing in memory or stored on disk.
3. **Query Processor** — Receives as input: a user query. Produces as output: a list of ranked document identifiers accompanied by their respective scores.

Data flow: `CSV data -> Tokenizer -> Indexer -> Query Processor`

1.2 Dataset

We use the Wikipedia Movies dataset from Kaggle (https://www.kaggle.com/datasets/exactful/wikipedia-movies). It contains approximately 17.8k movies across the 1970s–2020s. The CSV schema is `title,image,plot`. The `image` field is a URL and is not used for indexing; we index only the textual fields below:

- **title** (string)
- **plot** (string)

The document text is defined as `title + plot`. Each record is assigned a sequential integer document id at load time. Cleaning and normalization are handled by the tokenizer; the source CSVs are left unchanged.

2 Tokenizer

Tokenisation is the first layer of our pipeline and is shared by both document indexing and query processing, so that terms are always compared in the same space (`remove_stopwords=True`, `apply_stemming=True`).

2.1 NLTK-based pipeline

We rely on the Natural Language Toolkit (**NLTK**) https://www.nltk.org, a Python library for text processing. In particular, we use three standard NLTK components: `word_tokenize` (Punkt sentence tokenizer), `PorterStemmer`, and the English `stopwords` list. Starting from a raw plot, the processing flow is:

1. **Tokenisation**: `word_tokenize` splits the text handling punctuation and contractions better than a simple regex.
2. **Normalisation**: all tokens are lowercased and only alphanumeric items are kept.
3. **Stopword removal**: common English stopwords (e.g. *the*, *is*, *in*) are filtered out.
4. **Stemming**: `PorterStemmer` reduces inflected forms to a common stem (e.g. *detective*, *detectives*, *detecting* → *detect*).

Example:

"The detective investigates three murders in the city."

→ `[detect, investig, three, murder, citi]`

2.2 Design choices

Stopwords. We remove stopwords because: (i) they constitute a large fraction of tokens but carry little semantic content; (ii) BM25 and VSM already apply IDF weighting, and removing stopwords reduces index size and speeds up query processing with minimal loss in retrieval quality for most queries. For phrase or proximity queries that rely on word order (e.g. exact film titles), stopwords can be optionally retained in a separate field or module.

Numbers. We keep numerals since they convey useful signals in movie data (years, sequels, versions).

Effect on vocabulary. On our 17,830 plots, removing stopwords and enabling Porter stemming significantly reduces the vocabulary, grouping morphological variants and eliminating high-frequency common words. This improves both index efficiency and recall for content-bearing queries like "detective investigation".

For the general tokenisation principles we followed *Introduction to Information Retrieval*, ch. 2, in particular §§2.2–2.4.

3 Indexer

The indexer component implements three variants of inverted index construction, each suited for different use cases and scalability requirements.

3.1 Memory Index – SPIMI Approach

Reference: IIR §4.3

The memory index is based on the SPIMI (Single-Pass In-Memory Indexing) algorithm described in IIR §4.3. The book emphasizes that SPIMI avoids sorting and writes complete inverted blocks directly to memory: *"SPIMI generates a complete inverted index for a block without sorting the terms."*

In our implementation, each document is tokenized and inserted into a Python dictionary:

```
index = { term: { doc_id: tf } }
```

The dictionary grows automatically as new terms appear, thanks to Python's hash-based data structure. This design keeps insertions roughly O(1) and enables quick term lookups.

Because our dataset is small, we keep the entire index in memory. For larger collections, as explained in IIR Figure 4.4, SPIMI can be extended to write blocks to disk and merge them later. The lecture slides also visualized this "block → partial index → final merge" workflow step by step.

3.2 Disk Index – Term-per-File and Lazy Loading

Reference: IIR §5.2, §5.3

The disk version focuses on storing the index persistently. Each term is stored in a separate file, making it easy to test and inspect. To avoid naming conflicts, we used MD5 hashing for term filenames:

```
path = index_dir + "/terms/" + hashlib.md5(term.encode()).hexdigest() + ".pkl"
```

Alongside term files, we keep a `lexicon.pkl` file with metadata such as `df` (document frequency), `cf` (collection frequency), and file path. This matches the structure described in IIR §5.2: *"An inverted index consists of a dictionary and a set of postings lists stored on disk."*

When querying, only the relevant term's file is loaded — this lazy-loading design aligns with the assignment goal of reading only the necessary index parts into memory. In class, this was highlighted under the slide "Efficient Index Storage and Access."

3.2.1 Compression Implementation

Reference: IIR §5.3 To make the disk index smaller, we implemented both **gap encoding** and **Variable-Byte (VB)** compression. According to IIR §5.3, "gap encoding followed by variable byte coding gives a good balance between compression and speed." Our implementation compresses each postings list as follows:

```
def _compress_postings(postings):
    gaps = _gap_encode(postings)
    compressed = bytearray()
    for gap, tf in gaps:
        compressed.extend(_vb_encode(gap))
        compressed.extend(_vb_encode(tf))
    return bytes(compressed)
```

This combination reduced file size noticeably while keeping decoding fast. We chose VB instead of gamma codes because it is simpler, faster to decode in Python, and more suitable for small to mid-size datasets — as also explained in the "Compression Techniques" lecture slide.

3.3 Updatable Index – Auxiliary Index and Merge

Reference: IIR §4.5

The third mode adds support for updates, insertions, and deletions. We followed the dynamic indexing model from IIR §4.5, which recommends keeping a small auxiliary index in memory and merging it periodically with the main on-disk index. This approach supports incremental updates efficiently without rebuilding the entire index.

3.3.1 Structure and Components

The updatable indexer maintains three main parts:

- **Main on-disk index:** The compressed postings built earlier.

- **Auxiliary in-memory index (`aux`):** Stores new or modified documents before merging.
- **Tombstone set (`deleted`):** Tracks document IDs marked as deleted.

Initialization example:

```
def init_upd(index_dir="updindex", tokenizer=None):
    base = init_disk(index_dir, tokenizer)
    load_disk_min(base)
    base.update({
        "aux": defaultdict(dict),
        "deleted": set()
    })
    return base
```

3.3.2 Adding and Deleting Documents

New documents are first indexed into the auxiliary memory index:

```
def add_upd(st, title, text):
    did = st["next_id"]
    st["next_id"] += 1
    toks = re.findall(r"[a-z0-9]+", text.lower())
    for t, tf in Counter(toks).items():
        st["aux"][t][did] = tf
    return did
```

To delete a document, we simply mark it using a tombstone (instead of rewriting the entire index immediately):

```
def delete_upd(st, doc_id):
    st["deleted"].add(doc_id)
```

This idea matches the explanation in IIR §4.5: *"We maintain a small auxiliary index in memory and occasionally merge it with the main index."* In the lecture on dynamic indexing, this approach was illustrated as *"Fast updates using auxiliary memory and periodic merges."*

3.3.3 Merging Step

When a merge is triggered, the auxiliary postings are merged with the main index, deleted documents are removed, and the postings are re-compressed:

```
def merge_upd(st):
    for term in st["aux"].keys():
        main = postings_disk(st, term) if term in st["lex"] else {}
        main.update(st["aux"][term])
```

```
        for d in list(main.keys()):
            if d in st["deleted"]:
                del main[d]
        compressed = _compress_postings(main)
        with open(_term_path(st["dir"], term), "wb") as f:
            f.write(compressed)
```

This mirrors the merge mechanism described in both the book (IIR §4.5) and the "Dynamic Indexing" lecture slides.

3.3.4 Design Rationale

We decided not to implement the more complex *logarithmic merging* model (IIR §4.5) because it is meant for large-scale systems. Our single-level merge model is simpler and demonstrates the same concept clearly for small datasets. Logarithmic merging reduces the overall update cost from $O(T^2)$ to $O(T \log(T/n))$, but for this project, simplicity and clarity were our priorities.

3.3.5 Implementation Outcome

This version completes the full indexer lifecycle — from initial indexing to compression and dynamic updates — fully reflecting the concepts taught in both the book and the course slides.

4 Query Processing

Query processing analyzes the user's text, maps it into the same token space used for indexing, and ranks documents by estimated relevance. We implemented two standard models from IIR (ch. 6–7): the Vector Space Model (SMART weighting) and BM25. Both models use IDF, which down-weights frequent terms; this is the main reason we can safely retain stopwords in the tokenization stage.

4.1 Vector Space Model (SMART ltc.ltc)

Let N be the number of documents and df_t the document frequency of term t. We use the natural logarithm for inverse document frequency:

$$\mathrm{idf}_t = \ln\left(\frac{N}{df_t}\right).$$

For SMART `ltc.ltc`, we apply log-augmented TF and cosine normalization to both query and documents. For a term t with term frequency $tf_{t,x}$ in a vector $x \in \{q, d\}$ (query or document), the weight is

$$w_{t,x} = (1 + \ln tf_{t,x})\,\mathrm{idf}_t\,.$$

Vectors are then ℓ_2-normalized, and similarity is computed with cosine:

$$\text{score}(q, d) = \frac{\sum_{t \in q \cap d} w_{t,q}\, w_{t,d}}{\|q\|\, \|d\|}\,.$$

We also support `ntc.ltc`, which uses raw term frequency (no log) on the query side:

$$w_{t,q}^{\text{ntc}} = tf_{t,q}\, \text{idf}_t\,.$$

In both variants, IDF reduces the impact of very frequent terms; cosine normalization counteracts document-length effects typical of long movie plots.

4.2 BM25

BM25 further adjusts term contributions by document length with two parameters, k_1 and b. With dl the document length (in tokens) and $\overline{dl}$ the corpus average length, we use $k_1 = 1.5$ and $b = 0.75$:

$$\text{score}(q, d) = \sum_{t \in q} \text{idf}_t \cdot \frac{tf_{t,d}(k_1 + 1)}{tf_{t,d} + k_1 \left(1 - b + b\, \frac{dl}{\overline{dl}}\right)}\,.$$

For consistency with our VSM implementation, we use $\text{idf}_t = \ln(N/df_t)$; standard BM25 variants with $\ln(\frac{N - df_t + 0.5}{df_t + 0.5})$ are also common and can be substituted with minimal code changes. BM25 often surfaces documents whose length differs from the average, complementing cosine-normalized VSM.

5 Conclusion

This assignment implemented a full information retrieval pipeline over the Wikipedia Movies dataset, covering tokenization, indexing, and query processing. The system successfully indexed 17,830 movie documents with a vocabulary of 65,526 unique terms and 3,680,051 total postings, averaging 56.2 postings per term. This confirmed the efficiency of the SPIMI-based indexer and the correctness of the posting statistics.

The query processor integrated two retrieval models: BM25 and the Vector Space Model (VSM) using SMART `ltc.ltc` and `ntc.ltc` weighting. Both were tested on multiple queries to evaluate ranking quality. Results showed consistent and interpretable rankings for different query types. BM25 performed slightly better on descriptive, longer queries, while SMART weighting favored documents with higher term concentration.

Example Results

The following table summarizes the top retrieved titles for a few example queries.

```
[Method 1] SMART ltc.ltc (VSM)
--------------------------------------------------------------------------------
```

```
Query: "love love story romantic comedy"
    1. Can I Do It... 'Til I Need Glasses? (score: 0.4132)
    2. The Golden Boys (score: 0.2951)
    3. Folk Hero & Funny Guy (score: 0.2572)
    4. Pie in the Sky (1996 film) (score: 0.2445)
    5. Burning Annie (score: 0.2337)

Query: "space adventure alien planet"
    1. The Brother from Another Planet (score: 0.2918)
    2. Codependent Lesbian Space Alien Seeks Same (score: 0.2619)
    3. Metamorphosis: The Alien Factor (score: 0.2418)
    4. Final Days of Planet Earth (score: 0.2274)
    5. Star Raiders: The Adventures of Saber Raine (score: 0.2123)

Query: "murder mystery detective investigation"
    1. Final Combination (score: 0.2128)
    2. House of Bodies (score: 0.2050)
    3. One of Us Tripped (score: 0.1887)
    4. Murder She Purred: A Mrs. Murphy Mystery (score: 0.1815)
    5. Martial Law 2: Undercover (score: 0.1806)


[Method 2] SMART ntc.ltc (VSM)
--------------------------------------------------------------------------------
Query: "love love story romantic comedy"
    1. Can I Do It... 'Til I Need Glasses? (score: 0.4016)
    2. The Golden Boys (score: 0.2869)
    3. Folk Hero & Funny Guy (score: 0.2500)
    4. Pie in the Sky (1996 film) (score: 0.2377)
    5. Isn't It Romantic (2019 film) (score: 0.2291)

Query: "space adventure alien planet"
    1. The Brother from Another Planet (score: 0.2918)
    2. Codependent Lesbian Space Alien Seeks Same (score: 0.2619)
    3. Metamorphosis: The Alien Factor (score: 0.2418)
    4. Final Days of Planet Earth (score: 0.2274)
    5. Star Raiders: The Adventures of Saber Raine (score: 0.2123)

Query: "murder mystery detective investigation"
    1. Final Combination (score: 0.2128)
    2. House of Bodies (score: 0.2050)
```

```
    3. One of Us Tripped (score: 0.1887)
    4. Murder She Purred: A Mrs. Murphy Mystery (score: 0.1815)
    5. Martial Law 2: Undercover (score: 0.1806)


[Method 3] BM25 Ranking
--------------------------------------------------------------------------------
Query: "love love story romantic comedy"
    1. Isn't It Romantic (2019 film) (score: 20.5524)
    2. Just Another Romantic Wrestling Comedy (score: 17.3777)
    3. Big Ain't Bad (score: 16.5540)
    4. Punchline (film) (score: 16.1760)
    5. Butch Jamie (score: 15.0094)

Query: "space adventure alien planet"
    1. Codependent Lesbian Space Alien Seeks Same (score: 20.5140)
    2. Star Raiders: The Adventures of Saber Raine (score: 19.8484)
    3. Independence Day: Resurgence (score: 19.5832)
    4. Cosmic Sin (score: 19.4521)
    5. The Adventures of Captain Zoom in Outer Space (score: 19.3419)

Query: "murder mystery detective investigation"
    1. Massage Parlor Murders! (score: 15.1448)
    2. House of Bodies (score: 14.8520)
    3. Final Combination (score: 14.7663)
    4. Murder by Death (score: 14.5096)
    5. Exposed (2016 film) (score: 14.3995)
```

The results confirm that the retrieval component behaves consistently across different scoring schemes. BM25 tends to assign higher numeric scores and emphasizes document length normalization, while SMART weighting keeps values normalized through cosine similarity. Overall, the system returns relevant and explainable results, validating the effectiveness of the implemented ranking models.

References

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
Available at: https://nlp.stanford.edu/IR-book/

Calders, T., & Tokpo, E. (n.d.). *Information Retrieval: Index Construction and Maintenance*. Lecture slides, University of Antwerp.
Calders, T., & Tokpo, E. (n.d.). *Information Retrieval: Evaluation and Feedback Expansion*. Lecture slides, University of Antwerp.
University of Antwerp. (n.d.). *Information Retrieval: Boolean search and VSM*. Lecture slides.
University of Antwerp. (n.d.). *Information Retrieval: Other retrieval models*. Lecture slides.
Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. See also the NLTK project website: https://www.nltk.org.